Enabling People
Education for Future Generations

Curso de Inteligencia Artificial

Capítulo 3. Quiz

Para estudiantes

Nombre: Alfredo Avila Escobar

ⓒ2023 SAMSUNG. All rights reserved.
Samsung Electronics Corporate Citizenship Office holds the copyright of this document.

Samsung Innovation Campus

Enabling People
Education for Future Generations

This document is a literary property protected by copyright law so reprint and reproduction without permission are prohibited.
To use this document other than the curriculum of Samsung Innovation Campus, you must receive written consent from copyright holder.

1. Responde las siguientes preguntas usando funciones de la librería pandas.

```
import pandas as pd
import os

# Asegúrate de encontrarte en el directorio donde se encuentra el archivo.
# os.chdir(r'~~')     # Reemplaza el path con el tuyo.
df = pd.read_csv('data_studentlist.csv', header='infer')
df.head(5)
```

1) Altura promedio de los estudiantes varones:

1) Altura promedio de los estudiantes varones:

```
In [38]: altura_promedio = df.Height[df['Gender'].str.lower() == 'm'].mean(axis=0, skipna=True)
         print(altura_promedio)

         172.41000000000003
```

Respuesta: 172.41 cm

2) Altura promedio de las estudiantes:

2) Altura promedio de las estudiantes:

```
In [39]: altura_promedio_mujeres = df.Height[df['Gender'] == 'F'].mean(axis=0, skipna=True)
         print(altura_promedio_mujeres)

         166.64285714285714
```

Respuesta: 166.64 cm

3) Peso promedio de los estudiantes varones:

3) Peso promedio de los estudiantes varones:

```
In [40]: peso_promedio_hombres = df.Weight[df['Gender'].str.lower() == 'm'].mean(axis=0, skipna=True)
         print(peso_promedio_hombres)

         68.50000000000001
```

Respuesta: 68.50 kg

4) Peso promedio de las estudiantes:

4) Peso promedio de las estudiantes:

```
In [41]: peso_promedio_mujeres = df.Weight[df['Gender'].str.lower() == 'f'].mean(axis=0, skipna=True)
         print(peso_promedio_mujeres)

         50.44285714285714
```

Respuesta: 50.44 kg

5) La mayor estatura entre los estudiantes varones:

5) La mayor estatura entre los estudiantes varones:

```
In [42]: mayor_estatura_hombres = df.Height[df['Gender'].str.lower() == 'm'].max()
         print(mayor_estatura_hombres)

         182.1
```

Respuesta: 182.1 cm

6) La menor estatura entre las estudiantes:

6) La menor estatura entre las estudiantes:

```
In [43]: menor_estatura_mujeres = df.Height[df['Gender'].str.lower() == 'f'].min()
         print(menor_estatura_mujeres)
```

```
155.2
```

Respuesta: 155.2 cm

7) El peso más ligero entre los estudiantes varones de estatura superior a 1.75 m:

7) El peso más ligero entre los estudiantes varones de estatura superior a 1.75 m:

```
In [45]: peso_mas_ligero_altura = df.Weight[df['Height'] > 175 & (df['Gender'].str.lower() == 'm')].min()
         print(peso_mas_ligero_altura)
```

```
45.2
```

Respuesta: 61.3 kg

8) El mayor peso entre las estudiantes de estatura inferior a 1.60 m:

8) El mayor peso entre las estudiantes de estatura inferior a 1.60 m:

```
In [46]: peso_mayor_mujeres_altura = df.Weight[(df['Height'] < 160) & (df['Gender'].str.lower() == 'f')].max()
         print(peso_mayor_mujeres_altura)
```

```
45.3
```

Respuesta: 45.3 kg

9) La nota promedio de los alumnos (hombres y mujeres) sin ausencias ('N'):

9) La nota promedio de los alumnos (hombres y mujeres) sin ausencias ('N'):

```
In [48]: nota_promedio_sin_ausencias = df.Grade[df['Absence'] == 'N'].mean(axis=0, skipna=True)
         print(nota_promedio_sin_ausencias)
```

```
2.076923076923077
```

Respuesta: 2.08 (2.0769...)

10) La nota promedio de los alumnos (hombres y mujeres) con absentismo ('Y'):

10) La nota promedio de los alumnos (hombres y mujeres) con absentismo ('Y'):

```
In [49]: nota_promedio_ausencias = df.Grade[df['Absence'] == 'Y'].mean(axis=0, skipna=True)
         print(nota_promedio_ausencias)
```

```
3.0
```

Respuesta: 3.0

11) Altura promedio de los alumnos (hombres y mujeres) con grupo sanguíneo 'A' o 'AB':

11) Altura promedio de los alumnos (hombres y mujeres) con grupo sanguíneo 'A' o 'AB':

```
In [50]: altura_promedio_alumnos = df.Height[(df['Bloodtype'].str.lower() == 'a') | (df['Bloodtype'].str.lower() == 'ab')].mean(axis=0, s
         kipna=True)
         print(altura_promedio_alumnos)
```

```
172.64285714285714
```

Respuesta: 172.64 cm

12) Altura promedio de los alumnos varones con grupo sanguíneo 'A' o 'AB':

12) Altura promedio de los alumnos varones con grupo sanguíneo 'A' o 'AB':

```
In [51]: altura_promedio_alumnos_hombres = df.Height[(df['Bloodtype'].str.lower() == 'a') | (df['Bloodtype'].str.lower() == 'ab') & (df
         ['Gender'].str.lower() == 'm')].mean(axis=0, skipna=True)
         print(altura_promedio_alumnos_hombres)
```

```
173.0666666666667
```

Respuesta: 173.38 cm (173.375)

13) Edad promedio de los alumnos (hombres y mujeres) con absentismo ('Y') cuya nota es mayor o igual a 3:

13) Edad promedio de los alumnos (hombres y mujeres) con absentismo ('Y') cuya nota es mayor o igual a 3:

```
In [52]: edad_promedio_alumnos_ausentes = df.Age[(df['Absence'] == 'Y') & (df['Grade'] >= 3) ].mean(axis=0, skipna=True)
         print(edad_promedio_alumnos_ausentes)

         23.666666666666668
```

Respuesta: 23.67 años

2. Responde las siguientes preguntas usando funciones de las librerías pandas y numpy.

```
import pandas as pd
import numpy as np
import os

# Asegúrate de encontrarte en el directorio donde se encuentra el archivo.
# os.chdir(r'~~')     # Reemplaza el path con el tuyo.
df = pd.read_csv('data_census.csv', header='infer')
print(df.shape)
df.head(10)
```

1) Población total con ProvinceCode igual a 115 o 116:

Población total con ProvinceCode igual a 115 o 116:

```
In [54]: df['Population'] = df['Population'].str.replace(',', '', regex=False).astype(int)
         df['Households'] = df['Households'].str.replace(',', '', regex=False).astype(int)

In [55]: poblacion_total_provincecode = df.Population[(df['Province Code'] == 115) | (df['Province Code'] == 116)].sum()
         print(poblacion_total_provincecode)

         5970327
```

Respuesta: 5,970,327 habitantes

2) Población promedio de las ciudades donde hay más hombres (GenderRatio > 1):

Población promedio de las ciudades donde hay más hombres (GenderRatio > 1):

```
In [56]: poblacion_promedio_mas_hombres = df.Population[df['Gender Ratio'] > 1].mean()
         print(poblacion_promedio_mas_hombres)

         419221.2076923077
```

Respuesta: 419,221.21 habitantes en promedio

3) Lugares con más hombres (GenderRatio > 1) y menos de 2 personas por hogar:

Lugares con más hombres (GenderRatio > 1) y menos de 2 personas por hogar:

```
In [57]: lugares_mas_hombres_hogares_pequenos = df[(df['Gender Ratio'] > 1) & (df["Person's Household"] < 2)]
         lugares_mas_hombres_hogares_pequenos
```

Out[57]:

	Province Code	City Code	Population	Households	Person's Household	Gender Ratio
62	112	1155	20825	11172	1.86	1.29
149	101	1240	26790	13433	1.99	1.27
152	101	1017	30066	15485	1.94	1.15
167	117	1033	38027	19145	1.99	1.03
227	113	1120	43566	22248	1.96	1.11
253	104	1163	10304	5365	1.92	1.15

Respuesta: 6 ciudades cumplen la condición (GenderRatio > 1 y menos de 2 personas por hogar):

- Provincia 112, Ciudad 1155

- Provincia 101, Ciudad 1240

- Provincia 101, Ciudad 1017

- Provincia 117, Ciudad 1033

- Provincia 113, Ciudad 1120

- Provincia 104, Ciudad 1163

4) Ordena el DataFrame en orden ascendente de 'Households' y muestra los 10 primeros.

Ordena el DataFrame en orden ascendente de 'Households' y muestra los 10 primeros.

```
In [58]: df.sort_values('Households', ascending=True).head(10)
```

Out[58]:

	Province Code	City Code	Population	Households	Person's Household	Gender Ratio
253	104	1163	10304	5365	1.92	1.15
244	104	1147	17984	8769	2.05	0.97
200	114	1179	23308	10877	2.14	0.98
150	101	1130	24283	11162	2.18	1.17
62	112	1155	20825	11172	1.86	1.29
199	114	1073	25181	11723	2.15	0.98
241	104	1039	24096	12091	1.99	1.00
213	113	1034	27115	12465	2.18	0.95
198	114	1193	26325	12531	2.10	1.00
243	104	1208	26353	13233	1.99	0.98

Respuesta: las 10 ciudades con menos hogares (Provincia/Ciudad : Households):

104/1163 : 5,365 | 104/1147 : 8,769 | 114/1179 : 10,877 | 101/1130 : 11,162 | 112/1155 : 11,172

114/1073 : 11,723 | 104/1039 : 12,091 | 113/1034 : 12,465 | 114/1193 : 12,531 | 104/1208 : 13,233

3. Responde las siguientes preguntas.

```
import pandas as pd
import numpy as np
import os

# Asegúrate de encontrarte en el directorio donde se encuentra el archivo.
# os.chdir(r'~~')     # Reemplaza el path con el tuyo.
df = pd.read_csv('data_sales.csv', header='infer')
print(df.shape)
df.head(5)
```

1) Añade una nueva variable `Amount = UnitPrice x Units` y muestra las 5 primeras observaciones.

Añade una nueva variable Amount = UnitPrice x Units y muestra las 5 primeras observaciones.

```
In [60]: df['Amount'] = df['Unit Price'] * df['Units']
         df.head(5)
```

Out[60]:

	Date	Region	Item	Unit Price	Units	Amount
0	04-Jul-14	East	Pen Set	4.99	62	309.38
1	12-Jul-14	East	Binder	1.99	29	57.71
2	21-Jul-14	Central	Pen Set	12.49	55	686.95
3	29-Jul-14	East	Binder	19.99	81	1619.19
4	07-Aug-14	Central	Pen Set	23.95	42	1005.90

Respuesta: Amount = Unit Price × Units. Primeras 5 filas: 309.38, 57.71, 686.95, 1619.19, 1005.90

2) Calcula el precio unitario promedio de cada región. Utiliza el método `groupby()`.

Calcula el precio unitario promedio de cada región. Utiliza el método `groupby()`.

```
In [61]: precio_promedio_region = df.groupby('Region')['Unit Price'].mean()
         print(precio_promedio_region)
```

```
Region
Central    18.018750
East        9.143846
West       53.658333
Name: Unit Price, dtype: float64
```

Respuesta: Central 18.02, East 9.14, West 53.66

3) Calcula el precio unitario promedio de cada región. Utiliza el método `pivot_table()`.

Calcula el precio unitario promedio de cada región. Utiliza el método `pivot_table()`.

```
In [62]: precio_promedio_region = df.pivot_table(values='Unit Price', index='Region')
         print(precio_promedio_region)
```

```
            Unit Price
Region
Central    18.018750
East        9.143846
West       53.658333
```

Respuesta: Central 18.02, East 9.14, West 53.66 (mismo resultado que con groupby())

4) Calcula el precio unitario promedio para cada región en una línea de código. Utiliza el método `groupby()`.

Calcula el precio unitario promedio para cada región en una línea de código. Utiliza el método `groupby()`.

```
In [63]: print(df.groupby('Region')['Unit Price'].mean())
```

```
Region
Central    18.018750
East        9.143846
West       53.658333
Name: Unit Price, dtype: float64
```

Respuesta: df.groupby('Region')['Unit Price'].mean() -> Central 18.02, East 9.14, West 53.66

5) Calcula el precio unitario promedio y las unidades para cada región en una línea de código. Utiliza el método `pivot_table()`.

Calcula el precio unitario promedio y las unidades para cada región en una línea de código. Utiliza el método `pivot_table()`.

```
In [64]: print(df.pivot_table(values=['Unit Price', 'Units'], index='Region'))
```

```
         Unit Price      Units
Region
Central   18.018750  49.958333
East       9.143846  53.153846
West      53.658333  38.500000
```

Respuesta: (precio promedio, unidades promedio) : Central (18.02, 49.96), East (9.14, 53.15), West (53.66, 38.50)

6) Calcula el total de unidades para cada región y tipo de artículo en una línea de código. Utiliza el método `pivot_table()` y rellena los valores faltantes con 0.

Calcula el total de unidades para cada región y tipo de artículo en una línea de código. Utiliza el método `pivot_table()` y rellena los valores faltantes con 0.

```
In [65]: print(df.pivot_table(values='Units', index='Region', columns='Item', aggfunc='sum', fill_value=0))
```

```
Item     Binder  Desk  Pen  Pen Set  Pencil
Region
Central     424     7   27      243     498
East        234     0  175      152     130
West         64     3   76        0      88
```

Respuesta: tabla región × artículo con el total de unidades; p. ej. Central-Binder 424, East-Pen 175, West-Desk 3 (0 donde no hubo ventas).

7) Calcula el importa total de ventas para cada región y el tipo de artículo en una línea de código. Utiliza el método `pivot_table()` y rellena los valores faltantes con 0.

Calcula el importe total de ventas para cada región y el tipo de artículo en una línea de código. Utiliza el método `pivot_table()` y rellena los valores faltantes con 0.

```
In [67]: print(df.pivot_table(values='Amount', index='Region', columns='Item', aggfunc='sum', fill_value=0))
```

```
Item      Binder   Desk      Pen  Pen Set   Pencil
Region
Central  5762.63  875.0   539.73  2421.39  1540.32
East     2535.66    0.0  1354.25  1748.48   363.70
West     1279.36  825.0   151.24     0.00   231.12
```

Respuesta: tabla región × artículo con el importe total vendido; p. ej. Central-Binder 5,762.63, East-Pen 1,354.25, West-Pen Set 0.00 (no hubo ventas de ese artículo en West).

4. Responde las siguientes preguntas.

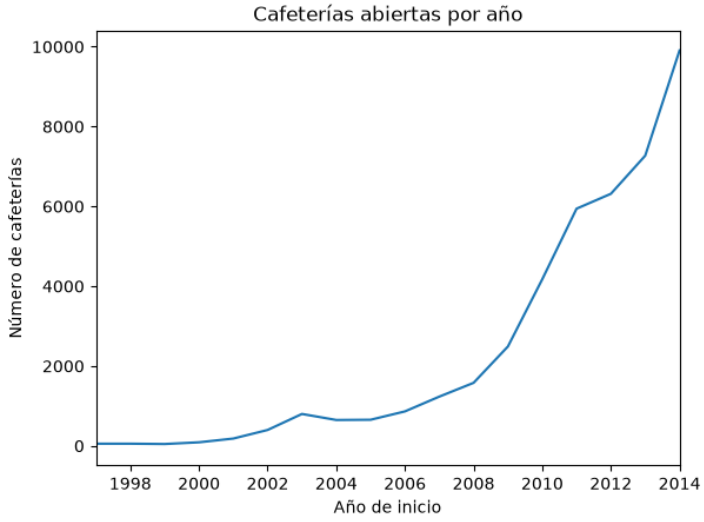
```
import pandas as pd
import numpy as np
import os
import matplotlib.pyplot as plt
%matplotlib inline

# Asegúrate de encontrarte en el directorio donde se encuentra el archivo.
# os.chdir(r'~~')     # Reemplaza el path con el tuyo.
df = pd.read_csv('data_coffeeshop.csv', header='infer', na_values=[' '])
print(df.shape)
df.head(5)
```

1) Genera una tabla de frecuencias para la variable `'yearOfStart'` y visualízala por año.

1) Genera una tabla de frecuencias para la variable `yearOfStart` y visualízala por año. Ordena por año. Dibuja una gráfica de línea de 1997 a 2014. *(Pista: `plt.xlim()`)*

```
In [69]: frecuencia_anual = df['Year of Start'].value_counts().sort_index()
         frecuencia_anual.plot(kind='line')
         plt.xlim(1997, 2014)
         plt.xlabel('Año de inicio')
         plt.ylabel('Número de cafeterías')
         plt.title('Cafeterías abiertas por año')
         plt.show()
```



Respuesta: las aperturas crecen poco hasta el año 2000 y se disparan después, sobre todo desde 2007 (ver gráfica).

- Ordena por año.
- Dibuja una gráfica de línea de 1997 a 2014. <= Pista: `plt.xlim()`

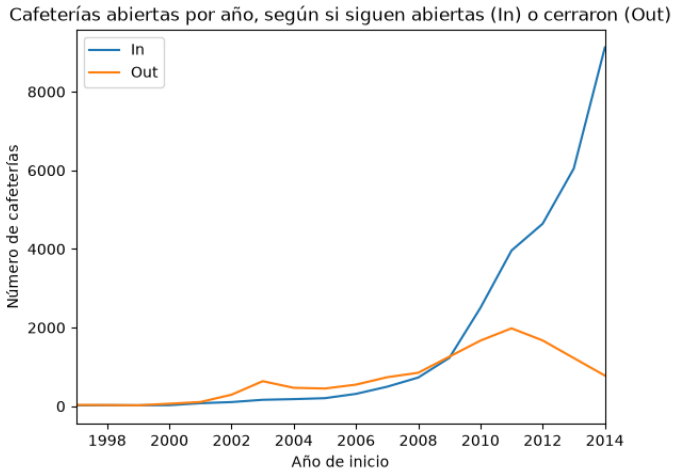
2) Divide los datos por el estado actual del negocio ('In' o 'Out'). Después, visualiza la tendencia annual de las frecuencias de la variable `'yearOfStart'`.

2) Divide los datos por el estado actual del negocio (`'In'` o `'Out'`). Después, visualiza la tendencia anual de las frecuencias de la variable `yearOfStart` . Ordena por año. Dibuja dos gráficas de líneas superpuestas de 1997 a 2014. Usa el objeto `figure` .

In [70]:
```
fig = plt.figure()

for estado in ['In', 'Out']:
    frecuencia = df[df['Current State'] == estado]['Year of Start'].value_counts().sort_index()
    plt.plot(frecuencia.index, frecuencia.values, label=estado)

plt.xlim(1997, 2014)
plt.xlabel('Año de inicio')
plt.ylabel('Número de cafeterías')
plt.title('Cafeterías abiertas por año, según si siguen abiertas (In) o cerraron (Out)')
plt.legend()
plt.show()
```



Respuesta: ver las dos líneas superpuestas (In vs Out) en la gráfica.

- Ordena por año.
- Dibuja dos gráficas de líneas superpuestas de 1997 a 2014.
- Usa el objeto `'figure'`.

3) A partir de los resultados de 1) y 2), ¿cómo describirías la tendencia en general?

Respuesta: Las aperturas casi no se mueven hasta el año 2000, y de ahí para adelante se disparan, sobre todo desde 2007-2008. Ya para 2013-2014 se abren miles de cafeterías al año. Separando por estado, las que siguen abiertas (In) no paran de subir, pero las que cerraron (Out) suben hasta como 2011 y después bajan. Esa bajada seguramente es porque los negocios más recientes todavía no han tenido tiempo de cerrar.

5. ¿Cuál es la sintaxis correcta para crear un arreglo de NumPy?
   ① `np.createArray([1, 2, 3, 4, 5])`
   ② `np.array([1, 2, 3, 4, 5])`
   ③ `np.object([1, 2, 3, 4, 5])`

Respuesta: B) np.array([1, 2, 3, 4, 5])

6. ¿Cuál de las siguientes opciones es un arreglo de dos dimensiones?
   ① `42`
   ② `([1, 2, 3], [4, 5, 6])`
   ③ `[1, 2, 3, 4, 5]`

Respuesta: B) ([1, 2, 3], [4, 5, 6])

7. ¿Cuál es la sintaxis correcta para comprobar el número de dimensiones de un arreglo?
   ① `np.dim()`
   ② `np.ndim()`
   ③ `np.ndim`
   ④ `np.dim`

Respuesta: C) np.ndim

8. ¿Cuál es la sintaxis correcta para imprimir el primer elemento de un arreglo?
   ① `print(myArr[0])`
   ② `print(myArr,1)`
   ③ `print(myArr[1])`

Respuesta: A) print(myArr[0])

9. ¿Cuál es la sintaxis correcta para imprimir el número 8 de la siguiente matriz?

```
arr = np.array([[1,2,3,4,5], [6,7,8,9,10]])
```

   ① `print(arr[3, 0])`
   ② `print(arr[7, 2])`
   ③ `print(arr[1, 2])`

Respuesta: C) print(arr[1, 2])

10. ¿Cuál es la sintaxis correcta para imprimir los números `[3, 4, 5]` de la siguiente matriz?

```
arr = np.array([1,2,3,4,5,6,7])
```

   ① `print(arr[2:4])`
   ② `print(arr[2:5])`
   ③ `print(arr[2:6])`
   ④ `print(arr[3:6])`

Respuesta: B) print(arr[2:5])

11. ¿Cuál es el método correcto para unir dos o más arreglos?
   ① `array_join()`
   ② `concatenate()`
   ③ `join()`

Respuesta: B) concatenate()

12. ¿Cuál es la sintaxis correcta para devolver el índice de todos los elementos que tienen el valor 4 de la siguiente matriz?

```
arr = np.array([1,4,3,4,5,4,4])
```

① `arr.search(4)`
② `arr.where()`
③ `np.where(arr==4)`

Respuesta: C) np.where(arr==4)

13. Cuando se utiliza el módulo `random` de NumPy, ¿cómo se puede devolver un número aleatorio entre 0 y 100?

① `random.rand(100)`
② `random.rand()`
③ `random.randint(100)`

Respuesta: C) random.randint(100)

14. Cuando se utiliza el módulo `random` de NumPy, ¿cómo se puede devolver una Distribución Normal de Datos con 1000 números, concentrados alrededor del número 50 y con una desviación estándar de 0.2?

① `random.normal(size=1000, loc=50, scale=0.2)`
② `random.normal(size=1000, mean=50, deviation=0.2)`
③ `random.normal(size=1000, normal=50, s=0.1)`

Respuesta: A) random.normal(size=1000, loc=50, scale=0.2)

15. ¿Cuál es la sintaxis correcta para sumar matemáticamente los números de `arr1` a los números de `arr2`?

① `np.add(arr1, arr2)`
② `np.append(arr1, arr2)`
③ `sum(arr1, arr2)`

Respuesta: A) np.add(arr1, arr2)

16. ¿Cuál es la sintaxis correcta para restar los números de `arr1` de los números de `arr2`?

① `np.min(arr1, arr2)`
② `np.minus(arr1, arr2)`
③ `np.sub(arr1, arr2)`
④ `np.substract(arr1, arr2)`

Respuesta: D) np.substract(arr1, arr2) [ojo: la opción tiene una errata; la función real es np.subtract]

17. ¿Cuál es el método correcto para redondear decimales en NumPy?

① `np.fix()`
② `np.trunc()`
③ Las otras 3 opciones son métodos válidos en NumPy
④ `np.around()`

Respuesta: C) Las otras 3 opciones son métodos válidos en NumPy

18. ¿Cuál sería la respuesta de esta suma acumulativa en NumPy?

```
arr = np.array([1,2,3])
print(np.cumsum(arr))
```

① [3, 6, 9]
② [6]
③ [9]
④ [1 3 6]

Respuesta: D) [1 3 6]

19. ¿Cuál es la sintaxis correcta para crear una Series de pandas a partir de una lista?

① pd.Series(mylist)
② pd.createSeries(mylist)
③ pd.getSeries(mylist)

Respuesta: A) pd.Series(mylist)